Problem A
A Little Leftover Pizza
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Greater New York Regional Programming Contest

The CS department has just had a big party and ordered too much pizza. Now it is time to put away the leftovers. They ordered a number of small, medium, and large pizzas, and there are still slices remaining in some or all of the pizza boxes. A small pizza comes in 6 slices, a medium pizza in 8 slices, and a large pizza in 12 slices. To save space, you can combine the leftover slices from the same size pizzas into a box of the right size, but you can't put a slice into a box for a different sized pizza, and you can't put more slices into a box than it originally held. What is the smallest number of boxes you will need to hold all the leftovers?

Input

The first line of input contains one positive integer n ($n < 1000$), the number of pizzas that were ordered. Each of the following n lines contains two items s_i and l_i (separated by a space) representing the leftovers for a given pizza. s_i is a string `S`, `M`, or `L` representing the size of pizza i, and l_i is an integer representing the number of leftover slices for pizza i. You can assume that each l_i is between zero and the original number of slices of that size pizza, inclusive.

Output

Output a single number, the fewest possible total boxes that can hold the leftover pizza according to the constraints given above.

Sample Input 1	Sample Output 1
3 S 0 M 5 L 0	1

Sample Input 2	Sample Output 2
3 S 3 S 4 S 2	2

Sample Input 3

```
4
S 1
M 1
M 3
L 1
```

Sample Output 3

```
3
```

Sample Input 4

```
4
L 6
M 2
M 6
L 6
```

Sample Output 4

```
2
```

Problem B
An Elephant Problem
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Pacific Northwest Regional Programming Contest

Gena is training an elephant to compete in the Iteratively Counting Peanuts Confrontation. As part of the training, Gena puts out a number of peanuts and the elephant has to count all of them.

However, elephants are not that good at counting peanuts, and Gena's elephant crushes some of the peanuts during the training. Gena is tasked with cleaning up the peanut debris that is left over.

Gena knows the weight of each peanut weighed in milligrams, but forgot how many peanuts he put out for his elephant. Although there could be up to $weight \times count$ milligrams of peanut debris, for various reasons the amount of debris could be much lower.

Help Gena compute the minimum number of peanuts he could have put out for the elephant!

Input

The first and only line of input contains two integers, m and d ($1 \leq m, d \leq 50$), where m is the weight of each peanut and d is the amount of peanut debris left over. Both quantities are in milligrams.

Output

Output a single integer, which is the minimum number of peanuts that Gena could have put out that could have resulted in his elephant leaving d milligrams of peanut debris.

Sample Input 1	Sample Output 1
1 1	1

Sample Input 2	Sample Output 2
3 14	5

Sample Input 3	Sample Output 3
27 18	1

Sample Input 4	Sample Output 4
6 7	2

Problem C
Baby Shark

Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Mid-Central USA Regional Programming Contest

Baby shark
Doo doo doo doo doo doo
Baby shark, doo doo doo doo doo doo
Baby shark, doo doo doo doo doo doo
Baby shark!

(Additional verses refer to Mommy Shark, Daddy Shark, Grandma Shark, Grandpa Shark, Let's Go Hunt, Run Away, Safe at Last, and It's the End.)

Notice that the word "doo" is repeated six times consecutively in lines 2, 3, and 4. (We ignore case, e.g., the uppercase "D" in line 2.)

Given a line of text, determine the maximum number of consecutive repetitions of a word. A *word* is defined as any sequence of characters separated by whitespace or by the start or end of the line.

Input

The input consists of a single line containing at least one word of English-like text composed of lowercase letters. Words are separated by single spaces, and spaces are not part of any word. The line contains at most 80 characters.

Output

Print the word, in all lowercase, that occurs consecutively the most times. If multiple words are tied for the maximum number of consecutive repetitions, print the **first such word in the input line** (not in alphabetical order).

Sample Input 1	Sample Output 1
`baby shark doo doo doo doo doo doo`	`doo`

Sample Input 2	Sample Output 2
`baby shark`	`baby`

Sample Input 3	Sample Output 3
no no no i wont wont wont	no

Problem D
Breakout
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Mid-Atlantic Regional Programming Contest

Uh oh! After you gave a low rating to one of Merlin's riddles, he has taken revenge by locking you inside his magical dungeon! This dungeon can be modeled as a circle with n rooms arranged along its circumference. The rooms are labeled from 1 to n in clockwise order.

You start at room 1 and know that there exists an exit in one of the rooms. To get to the exit room, you can walk either clockwise or counterclockwise to a neighboring room. However, Merlin is smart. To impede your escape, Merlin has placed wooden crates in several hallways connecting two adjacent rooms. To pass through a hallway, you must break any wooden crates blocking it.

You want to make it out to the exit room as quickly as possible. What is the minimum number of crates you must break through to escape?

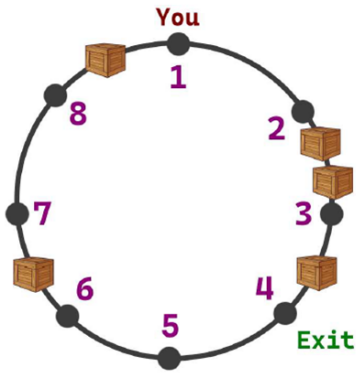


Figure D.1: Illustration of the sample case.

Input

The first line of input contains three integers n, x, and m ($2 \leq n \leq 1\,000$, $1 \leq x \leq n$, $0 \leq m \leq 1\,000$), meaning that there are n rooms with room x being the exit room, and there are m crates.

Each of the next m lines contains a single integer r ($1 \leq r \leq n$), indicating that there is a crate in the hallway clockwise between room r and the next room along the circumference. If $r = n$, it means that the crate is in the hallway between room n and room 1. These integers may contain duplicates, meaning that multiple crates are placed in the same hallway.

Output

Output a single integer, the minimum number of crates you must destroy to breakout.

Sample Input 1	Sample Output 1
8 4 5 2 6 3 2 8	2

Problem E
Fractional Sequence
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Northeast North America Regional Programming Contest

Consider the following increasing sequence, S, of rational numbers:

$$1,\ 2,\ 2\frac{1}{2},\ 3,\ 3\frac{1}{3},\ 3\frac{2}{3},\ 4,\ 4\frac{1}{4},\ 4\frac{1}{2},\ 4\frac{3}{4},\ 5,\ 5\frac{1}{5},\ 5\frac{2}{5},\ 5\frac{3}{5},\ 5\frac{4}{5},\ 6,\ \ldots.$$

S is composed of an infinite set of blocks, $N_1, N_2, N_3, \ldots$, where block N_i is

$$i,\ \ i+1/i,\ \ i+2/i,\ \ \ldots,\ \ i+(i-1)/i.$$

So $S(1) = 1, S(2) = 2, S(3) = 2\frac{1}{2}$, etc. Write a program which takes as input an integer n and outputs $S(n)$.

Input

Input is a single line containing an integer, n ($1 \leq n \leq 4 \cdot 10^9$).

Output

Output $S(n)$ as a single integer if the answer is a whole number. Otherwise, output the integer part, a single space and a proper fraction a/b in lowest terms (i.e. $0 < a < b$ and $GCD(a, b) = 1$). See the sample outputs.

Sample Input 1	Sample Output 1
326	26

Sample Input 2	Sample Output 2
448	30 2/5

Sample Input 3	Sample Output 3
4000000000	89443 19596/89443

This page is intentionally left blank.

Problem F
Heist of the Century
Time Limit: 4 seconds

After planning the most elaborate heist to steal The Crown Jewel of Count Monte Carlo, you have encountered the final obstacle in your path: a locked safe. However, you have trained for this very moment and honed your safecracking abilities.

The safe has N dials, each of which can be set to any integer value from 1 to $2N$. Count Monte Carlo has configured the safe with a correct, secret value for each dial. To attempt to open the safe, you set each dial to a value of your choice, and then turn the safe door handle. If every single dial is set to its correct secret value, you will feel no resistance and the door will swing open immediately.

Of course, opening the safe by randomly guessing all of the right secret values is highly unlikely to succeed. However, as an expert safecracker, even if your guess is wrong you feel some resistance when you attempt to open the door and you can use that knowledge to help decipher the correct secret values. If a dial has secret value h and the dial is set to d when you try to open the door, the dial will exert resistance $|h - d|$ to turning the door handle. You can feel the *maximum* resistance over all of the dials. (Note that if this value is 0, you have successfully opened the safe and completed your heist!)

Unfortunately, the security team has been made aware of your presence and they are closing in on your location. You are able to make one attempt at opening the safe per second, but they are $4N$ seconds away! Can you complete the heist before you get caught?

Interaction

This is an interactive problem. Interaction begins by reading an integer N ($1 \leq N \leq 500$) from standard input, the number of dials on the safe. Your program may make up to $4N$ attempts to open the safe door by specifying a value to try for each safe dial. You will then be told the resistance you feel from the door handle after each attempt.

To make an attempt, print a line containing N space-separated integers $d_1, d_2, \ldots, d_N$ ($1 \leq d_i \leq 2N$), the values you propose for each dial. Then read a single integer from standard input representing the resistance that you felt from the door handle, $\max_i |h_i - d_i|$, where h_i is the (unknown to you) secret value of dial i. If the resistance is 0, you have opened the safe and your program should terminate. Otherwise, if you have attempts remaining, you may try again.

If you run out of attempts, your program should exit cleanly (though it will be judged incorrect for failing to crack the safe in time to escape the guards).

The secret value of each dial was configured by Count Monte Carlo in advance of the heist and will not change in response to your cracking attempts.

Notes

Do not forget to flush the output stream after each line that you print and to cleanly exit after the interaction is done. Please also make sure that you follow the above interaction protocol **exactly** regarding what information to print on which line of output: for example, if the protocol requires you to print a list of space-separated integers on a single line, the judge **will not** accept each integer on its own line.

If the judge receives invalid or unexpected input, it will print -1 and then immediately exit. Your program must detect this error report and cleanly exit in order to receive a Wrong Answer verdict. If your program blocks waiting for further interaction from the judge, or tries to interpret the -1 as a resistance value, you may receive a different rejected verdict (such as Time Limit Exceeded or Runtime Error) instead of Wrong Answer.

You have been provided with a command-line tool for local testing. The tool has comments at the top explaining its use.

Read	Sample Interaction 1	Write
3		
		1 1 1
5		
		3 4 5
1		
		4 5 6
0		

Problem G
Intervals
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC North Central North America Regional Programming Contest

A new streaming service, *My Stream*, has launched and wants to analyze user activity patterns. Each time a user logs on, the service records the hour they started streaming and the hour they finished.

For example, if a user started streaming at 3 a.m. and ended at 2 p.m., this interval would be recorded as starting at time 3 and ending at time 14, for a total duration of 11 hours.

Given n user intervals, each represented as a pair of integers $[a_i, b_i]$ where $0 \leq a_i < b_i \leq 24$, and an integer k, determine how many hours during the day had at least k users streaming simultaneously.

Input

- The first line contains two integers n and k ($1 \leq n \leq 1000$, $1 \leq k \leq n$) — the number of user intervals and the minimum number of users to count.
- Each of the next n lines contains two integers a_i and b_i ($0 \leq a_i < b_i \leq 24$), representing the start and end time of the i-th user interval.

Output

Print a single integer — the number of hours during which at least k users were streaming.

Sample Input 1	Sample Output 1
4 2 12 18 1 13 17 20 18 22	4

Sample Input 2	Sample Output 2
7 3 8 17 2 5 3 12 11 23 14 19 10 20 9 13	10

Problem H
Memories of Passport Control
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC South Central USA Regional Programming Contest

After getting your latest passport, you are back to traveling internationally. This time, officers at passport control have been stamping only the earliest unstamped pages in your passport, so you have not needed to apply for a new passport early.

Due to new international aviation laws, when you pass through passport control, one of two things happens: either a single page is stamped, or exactly k pages are stamped. You pass through passport control exactly once per trip.

Your memory is foggy, and you no longer remember how many trips you have taken. Compute the minimum number of trips you could have taken to end up with exactly the given number of stamped pages in your passport.

Input

The first and only line of input contains two integers k and s ($2 \leq k \leq 50$, $1 \leq s \leq 50$), where k is the number of pages passport control stamps when stamping multiple pages at once, and s is the total number of pages that have been stamped in your passport.

Output

Output a single integer, the minimum number of trips you could have taken. It can be proven that it is always possible to have exactly s stamped pages.

Sample Input 1	Sample Output 1
13 23	11

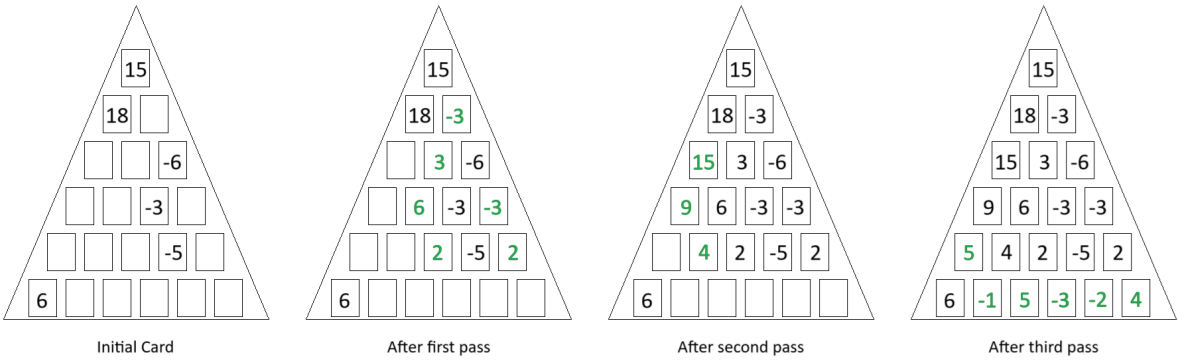
Sample Input 2	Sample Output 2
13 39	3

This page is intentionally left blank.

Problem I
Number Pyramid
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC East Central USA Regional Programming Contest

Uneedtoth, Inc. is a company that makes mathematical games for elementary school students, and Pam D. Monium has just been put in charge of their latest product: Number Pyramids. The game consists of cards each containing a pyramid of squares, some filled with numbers and some empty (see the first card on the left in Fig **??**). The object of the game is to find values that go in each of the empty squares so that the value in every square (except those on the bottom) is the sum of the values in the two squares below it. To keep things, well, elementary, the final values in the squares are always integers that are < 100 in absolute value. The cards are also set up so that they can be solved by multiple passes through the card, each pass going from the top to the bottom row and left to right in each row, filling in any value that can be determined by its neighbors. An example of this is shown in the remaining three cards in Fig **??**. Note that there may be other ways to solve a given card, but if a unique solution cannot be found using this method, the card should not be issued to the students. Pam calls cards like this "ambiguous".



Things seemed to be going smoothly for Pam, with machines in Uneedtoth's factory spitting out card after card. But when Pam started doing some product testing she noticed that there was a glitch in the machine and some of the cards were either ambiguous, or led to contradictory values for one or more squares. For example, if the card on the left in Fig **??** was missing the 6 in the lower left corner, that card would be ambiguous as it would allow multiple solutions; if that card had a 5 in the rightmost square in the last row, or a 3 in the rightmost square in the second from the top row, each of these would lead to a contradiction and the card would have no solution. Finally, if the 18 was changed to a -90, there would be no solution as one of the values solved for would be outside of the range $-99, \ldots, 99$

As Pam sees it, there are two paths she could take now. The first is to ignore the testing and ship out the cards, relying on American children's terrible math skills to give her enough time to find another job; or she can have someone write software to detect the bad cards and remove them.

After much soul searching and a look at her bank account she's decided on the latter and has come to you for help.

Input

The input begins with a line n ($2 \leq n \leq 100$) which is the number of rows on a pyramid card. Following this are n lines specifying the rows of the pyramid. The first line containing a single value, the second line two values, and so on. All values v will be integers in the range $-99 \leq v \leq 100$, where the special value of 100 indicates an empty square.

Output

Output one of three things: if the card has multiple solutions with all values in the range $-99, \ldots, 99$, output `ambiguous`; if the card has no solution, or has a solution with one or more values outside the range $-99, \ldots, 99$, output `no solution`; otherwise output `solvable` and then output the solution using the format shown in the Sample Output.

Sample Input 1

```
6
15
18 100
100 100 -6
100 100 -3 100
100 100 100 -5 100
6 100 100 100 100 100
```

Sample Output 1

```
solvable
15
18 -3
15 3 -6
9 6 -3 -3
5 4 2 -5 2
6 -1 5 -3 -2 4
```

Sample Input 2

```
2
100
10 100
```

Sample Output 2

```
ambiguous
```

Sample Input 3

```
2
10
10 10
```

Sample Output 3

```
no solution
```

Sample Input 4	Sample Output 4
2 100 51 51	no solution

This page is intentionally left blank.

Problem J
Penniless in America
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Southern California Regional Programming Contest

In May of 2025, the U. S. Treasury placed its last order for penny blanks (metal discs that have not yet been stamped into pennies). Production of pennies ceased in November 2025. Item prices in stores can continue to be posted to the cent, and electronic transactions will charge to the cent.

However, as pennies disappear from circulation, cash transactions will need to be rounded to the nearest nickel (five cents). A common rounding rule is as follows: If the final digit of a purchase ends in 3, 4, 8, or 9 cents, the total will be rounded up; if it ends in 1, 2, 6, or 7 cents, it will be rounded down. Transactions ending in 0 or 5 cents are not rounded. Your program must read a series of dollar and cent values, then print each amount rounded to the nearest five cents.

Input

The first line of input contains a single integer n ($1 \leq n \leq 100$) representing the number of transactions.

Each of the next n lines contains a transaction amount, expressed as dollars and cents (dollars, a decimal point, and exactly two digits after the decimal point). Each transaction amount is between 0.03 and 10000.00 inclusive.

Output

For each transaction amount, print a line with the transaction amount rounded to the nearest five cents. Express the value as dollars and cents (dollars, a decimal point, and two digits after the decimal point for cents, without a dollar sign or other punctuation). For amounts under one dollar, print a leading zero before the decimal point.

Sample Input 1	Sample Output 1
5 0.03 1.24 1.22 99.99 100.00	0.05 1.25 1.20 100.00 100.00

This page is intentionally left blank.

Problem K
Snakey String
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Southeast USA Regional Programming Contest

A *snakey string* is a fancy rendering of an otherwise normal string of text. When a string is made snakey, it is placed on a 2D grid such that the following conditions are met:

- The first character of the string is on the first column of the grid, the second on the second column, . . . , and the last on the last column.
- Each character in the string occupies either the row directly above or directly below the row of the previous character.

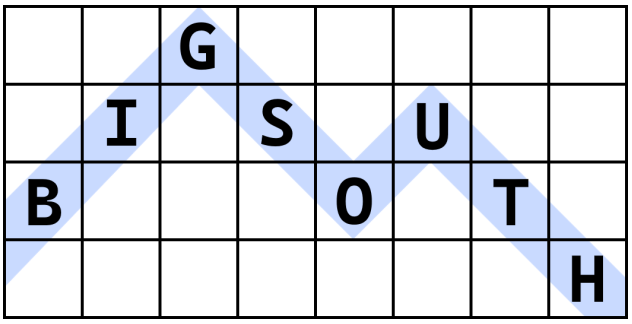


Figure K.1: The snakey string in the sample case.

Given a snakey string, can you recover the original string?

Input

The first line of input contains two integers r and c ($2 \leq r, c \leq 100$), the number of rows and columns of the grid.

The next r lines each contain c characters that form the 2D grid containing the snakey string. Empty cells are encoded with a period, while uppercase letters (`A-Z`) represent characters in the original string. Every column in the grid contains exactly one uppercase letter. It is possible that some rows do not contain any uppercase letters.

Output

Output a single string, the original string that was used to build this snakey string.

Sample Input 1	Sample Output 1
4 8 ..G..... .I.S.U.. B...O.T. ....H	BIGSOUTH

Problem L
Video Games
Time Limit: 1 second , Memory limit: 2G

From the 2025 ICPC Rocky Mountain Regional Programming Contest

Alice, Bob, and Charlie share a collection of video game cartridges. This way, only one of them needs to own a game in order for them all to enjoy it. The popular games right now are a fishing game, a golf game, and a hockey game.

Initially, Alice has the fishing game, Bob has the golf game, and Charlie has the hockey game. Over time, the games gets passed around between them and it can be hard to keep track of who currently has each game.

Given a sequence of requests indicating that a particular friend wants to play a particular game, determine who they have to borrow that game from.

Input

The first line of input contains a single integer n ($1 \leq n \leq 100$) indicating the number of requests.

Each of the next n lines contain a string describing a request in the form: a `wants to play` b where a is one of `alice`, `bob`, or `charlie` and b is one of `fishing`, `golf`, or `hockey`.

After each request, friend a is now in possession of game b.

Output

For each request, output a single line containing one of the following messages. If a was already in possession of game b, then output the message "a `already has` b" (without quotes). If a was not already in possession of b then output the message "a `borrows` b `from` c" (again without quotes) where c is the name of the friend who was in possession of game b at this time.

Sample Input 1

```
4
alice wants to play golf
bob wants to play golf
charlie wants to play hockey
charlie wants to play fishing
```

Sample Output 1

```
alice borrows golf from bob
bob borrows golf from alice
charlie already has hockey
charlie borrows fishing from alice
```